27. b. Singly Linked List – TraverseListReverseOrderUsingStack – Space Complexity and Algorithm.

***Program*: –**

```
#include <stack>
//#include <bits/stdc++.h>
…..
void TraverseListReverseOrderUsingStack(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    stack<Node *> s;
    Node **current = head;
    while (*current != nullptr)
    {
        s.push(*current);
        current = &((*current)->next);
    }
    while (!s.empty())
    {
        cout << s.top()->data << " ";
        s.pop();
    }
    cout << endl;
}
…case 25: {
            cout << endl;
            cout << endl;
            cout << "Traverse List Reverse Order Iterative Using Stack" << endl;
            TraverseListReverseOrderUsingStack(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

Space Complexity Analysis

1. *Input Space Complexity*

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, O(1).***

Thus, the total input space complexity is: O(1).

• head (type Node**) → one pointer-to-pointer (one machine word).

This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

B. Full – data (inclusive) Definition:

At call entry, *head may reference a list of n nodes. Those n Node objects dominate the input footprint, so storage required is proportional to n.

• Same single word as above (Θ(1)), plus

• Every node reachable from *head when the routine is called.

If the list has n nodes, they occupy Θ(n) bytes.⇒ Θ(n) = O(n)

This more common definition includes the entire data structure. If the linked list has **`n` nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

2. *Auxiliary Space Complexity*

Auxiliary space complexity refers to the extra space used by the algorithm that is not part of the input. In the case of the TraverseListReverseOrderUsingStack function, the auxiliary space is determined by the variables declared within the function:

What the algorithm itself allocates beyond its input:

• std::stack<Node *> s

– The stack object itself lives on the call stack → Θ(1).

– While traversing, each of the n node addresses is pushed; the underlying container (deque/vector) therefore allocates storage for n pointers → Θ(n).

• Local variables

– Node **current and a few compiler-generated temporaries → Θ(1).

• No recursion, no other dynamic allocations.

⇒ Total auxiliary space in the worst case: Θ(n) = O(n)

• std::stack<Node*> s

– The object itself is a tiny constant-size wrapper (O(1)), **but** it grows internally to hold one pointer for every node pushed.

– After the first while-loop completes, the stack contains n pointers → Θ(n) bytes.

• Node **current – one pointer-to-pointer (O(1)).

• A few compiler-generated temporaries / return address (O(1)).

• No recursion and no other dynamic allocations.

Because the stack's size increases linearly with the number of list elements, the auxiliary space required by the algorithm is linear:

Auxiliary space = Θ(n)

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function, the stack is the dominant factor.

1. **`stack<Node *> s`**: The function declares a stack to hold pointers to the list's nodes.
2. **`Node **current`**: A single pointer-to-a-pointer is declared for traversal, which takes up constant space.

3. **Stack Population**: The first `while` loop traverses the entire linked list. For each of the **`n` nodes** it visits, it pushes a pointer onto the stack `s`.
4. **Total Space**: After the first loop, the stack will contain `n` pointers. The space required for the stack is therefore directly proportional to the number of nodes in the list. The memory usage grows linearly with the input size.

Because the stack's size depends directly on the length of the list, the auxiliary space complexity is **Θ(n)**.

$\boldsymbol{Algorithm}$

$\boldsymbol{TraverseListReverseOrderUsingStack(START)}$

$\mathbf{1}.\,[START\ is\ NULLPTR?\,]if\ START = NULL, then:$

$Write: ``\ Error: List\ is\ Empty.``$

$Return\ and\ exit.$

$\mathbf{2}.\,STACK < TYPE\ PTR > S\ [Initialize\ an\ empty\ STACK\ named\ S\ which\ take\ pointer\ type\ variable]$

$[OR\ we\ \ can\ write\ here \rightarrow DECLARE\ S:\ \ STACK\ OF\ PTR\ ]$

$\mathbf{3}.\,SET\ CURRPTR := \ START$

$\mathbf{4}.\,Repeat\ while\ CURRPTR\ \neq NULL:$

$PUSH(S, CURRPTR)$

$- -$

$Translating\ \ PUSH(S, CURRPTR)$

$\mathbf{1}.\,[Stack\ already\ filled?\,]If\ TOP = MAXSTK, then:$

$Write: ``OVERFLOW``$

$Return\ and\ Exit.$

$\mathbf{2}.\,SET\ TOP := TOP + \mathbf{1}\ [INCREASES\ TOP\ by\ \mathbf{1}]$

$\mathbf{3}.\,SET\ S[TOP] := CURRPTR\ \begin{bmatrix} Inserts\ Addresses\ of\ Nodes\ pointed\ by\ CURRPTR \\ in\ \ TOP\ position\ of\ STACK. \end{bmatrix}$

$\mathbf{4}.\,Return.$

$- -$

$SET\ CURRPTR := ADDRESS\ of\ LINK[CURRPTR]$

$[END\ of\ Loop]$

$\mathbf{5}.\ \boldsymbol{Repeat\ while\ S[TOP] \neq -1}\ [\boldsymbol{STACK\ is\ not\ EMPTY}?\ ]:$

$\boldsymbol{Write}: \boldsymbol{INFO}[\boldsymbol{S}[\boldsymbol{TOP}]]$

$\boldsymbol{POP}(\boldsymbol{S})$

$- -$

$\boldsymbol{Translating\ \ POP}(\boldsymbol{S})$

$\mathbf{1}.\ [\boldsymbol{STACK\ has\ an\ item\ to\ be\ \ removed}?\ ]\boldsymbol{If\ TOP} = \mathbf{0}, \boldsymbol{then}:$

$\boldsymbol{Write}:$ ``$\boldsymbol{UNDERFLOW}$``

$\boldsymbol{Return\ and\ Exit}.$

$\mathbf{2}.\boldsymbol{SET\ TOP} := \boldsymbol{TOP} - \mathbf{1}\ [\boldsymbol{DECREASE\ TOP\ by}\ \mathbf{1}.\ ]$

$\mathbf{3}.\boldsymbol{Return}.$

$- -$

$[\boldsymbol{END\ of\ Loop}]$

$[\boldsymbol{END\ of\ Algorithm}]$